

DbDocGen User Guide

Database Documentation Generator v2.0

Table of Contents

1. [Overview](#)
 2. [Supported Databases](#)
 3. [Installation](#)
 4. [Using the GUI](#)
 5. [Connection Settings by Database Type](#)
 6. [Documentation Options](#)
 7. [Output Files](#)
 8. [ER Diagram Export](#)
 9. [Sample SQL Queries](#)
 10. [Custom SQL Query Executor](#)
 11. [PDF Generation](#)
 12. [Command-Line / API Usage](#)
 13. [Troubleshooting](#)
-

Overview

DbDocGen reverse-engineers a live database and produces professional documentation in **Markdown**, **HTML**, and **PDF** formats. It extracts tables, views, columns, data types, primary and foreign keys, indexes, constraints, and relationships, then renders an Entity-Relationship (ER) diagram alongside a complete data dictionary — all from a single GUI window or a few lines of Python.

Supported Databases

Database	Driver	Default Port
PostgreSQL	<code>psycopg2</code>	5432
Microsoft SQL Server	<code>pyodbc</code> + ODBC Driver 17/18	1433
Oracle	<code>cx_Oracle</code> + Instant Client	1521
IBM Informix	<code>pyodbc</code> + Informix ODBC Driver	9088
Microsoft Dataverse / Dynamics 365	<code>msal</code> + REST API	— (cloud)

Installation

Prerequisites

- Python 3.7 or higher
- pip

1. Install Core Dependencies

```
pip install markdown
```

2. Install Your Database Driver

Install only the driver(s) you need.

PostgreSQL

```
pip install psycopg2-binary
```

SQL Server

```
pip install pyodbc
```

Then install the [Microsoft ODBC Driver 17 for SQL Server](#) from Microsoft.

Oracle

```
pip install cx_Oracle
```

Then install [Oracle Instant Client](#) and add it to your `PATH`.

Informix

```
pip install pyodbc
```

Then install the [IBM Informix ODBC Driver](#).

Dataverse / Dynamics 365

```
pip install msal requests
```

3. Install PDF Support (Optional)

Recommended — wkhtmltopdf

```
pip install pdfkit
```

Download and install the wkhtmltopdf binary from wkhtmltopdf.org/downloads.html.

Alternative — WeasyPrint

```
pip install weasyprint
```

WeasyPrint requires GTK3 runtime libraries on Windows.

4. Install Excel Export Support (Optional)

```
pip install pandas openpyxl
```

Full Installation (all features)

```
pip install markdown weasyprint pycopg2-binary pyodbc cx_Oracle msal requests pandas openpyxl pdfkit
```

Using the GUI

Launch the application from the `DbDocGen` directory:

```
python dbdocgen.py
```

The GUI walks you through five steps:

Step 1 — Select Database Type

Use the radio buttons at the top to choose your database:

`PostgreSQL` · `SQL Server` · `Oracle` · `Informix` · `Dataverse`

The port field updates automatically when you change the type.

Step 2 — Enter Connection Details

Fill in the connection fields (see [Connection Settings by Database Type](#) for specifics).

Step 3 — Test Connection

Click **Test Connection**. A success or failure message confirms whether the application can reach the database before you generate anything.

Step 4 — Select Documentation Options

Enable or disable what to include in the output:

Option	Description
Include Tables	Document all tables with columns, types, and constraints
Include Views	Document all views including SQL definition
Include Indexes	List all indexes per table
Include Constraints	Include check and unique constraints
Include Relationships	Document foreign key relationships
Include ER Diagram	Embed a Mermaid ER diagram

Step 5 — Choose Output Folder and Formats

1. Click **Browse** and select the folder where files will be saved.
2. Check **Export as HTML** (recommended — always enabled by default).
3. Check **Export as PDF** if PDF output is needed.

Step 6 — Generate

Click **Generate Documentation**. A status bar shows progress. When complete, the output folder contains your documentation files.

Connection Settings by Database Type

PostgreSQL

Field	Value
Host	Server hostname or IP (e.g., <code>localhost</code>)
Port	<code>5432</code>
Database	Database name
Schema	Leave blank to use <code>public</code>
Username / Password	Database credentials

SQL Server

Field	Value
Host	Server name or IP
Port	1433
Database	Database name
Schema	Leave blank to use dbo
Windows Authentication	Check to use the current Windows login (no password needed)
ODBC Driver Version	Select 17 or 18 to match your installed ODBC driver

Oracle

Field	Value
Host	Server hostname or IP
Port	1521
Database	Service name or SID
Schema	Owner/schema name (defaults to current user)
Username / Password	Oracle credentials

Informix

Field	Value
Host	Server hostname or IP
Port	9088
Database	Database name
Username / Password	Informix credentials

Dataverse / Dynamics 365

Dataverse uses OAuth2 rather than a traditional connection string. You need an Azure AD app registration with Dataverse permissions.

Field	Value
Environment URL	Your org URL, e.g., <code>https://yourorg.crm.dynamics.com</code>
Client ID	Azure AD application (client) ID
Client Secret	Azure AD client secret value
Tenant ID	Azure AD directory (tenant) ID
Table Limit	Max entities to process (<code>0</code> = all)

See `DATAVERSE_SETUP.md` for instructions on creating the Azure AD app registration and Dataverse application user.

Documentation Options

Option	What It Generates
Tables	Column names, data types, nullability, defaults, PK/FK markers
Views	Column list plus the full SQL view definition
Indexes	Index name, indexed columns, and uniqueness flag
Constraints	Named check constraints and unique constraints
Relationships	Full foreign key relationship list with referenced table/column
ER Diagram	Mermaid <code>erDiagram</code> block embedded in HTML/Markdown

Output Files

After generation, the output folder contains:

File	Description
<code>{database}_documentation.md</code>	Markdown source — human-readable and version-control friendly
<code>{database}_documentation.html</code>	Styled, interactive HTML with the embedded ER diagram
<code>{database}_er_diagram.mmd</code>	Standalone Mermaid diagram file (editable)
<code>{database}_documentation.pdf</code>	PDF export (only if PDF option was selected)

ER Diagram Export

Every generation produces a `{database}_er_diagram.mmd` file containing your database structure as a Mermaid `erDiagram`. You can open and customize it in several ways:

Mermaid Live Editor

1. Open mermaid.live
2. Paste the contents of the `.mmd` file into the editor
3. Edit, style, and export as PNG, SVG, or PDF

GitHub / GitLab

Add the file to your repository — both platforms render `.mmd` files automatically in the file browser.

VS Code

Install the **Markdown Preview Mermaid Support** extension, then open the file for a live preview.

Example diagram format

```
erDiagram
    customers {
        int customer_id PK
        varchar name
        varchar email
    }
    orders {
        int order_id PK
        int customer_id FK
        date order_date
        decimal total
    }
    customers ||--o{ orders : places
```

Sample SQL Queries

The generated HTML documentation includes a **Sample Queries** section for every table with five ready-to-run templates:

Query	Purpose
<code>SELECT</code> (all rows)	Retrieve every record in the table
<code>SELECT</code> (by PK)	Retrieve a single record using the primary key
<code>INSERT</code>	Add a new record with all column names listed
<code>UPDATE</code>	Modify an existing record by primary key
<code>DELETE</code>	Remove a record by primary key

Replace the `<value>` placeholders with real values, then run the query in pgAdmin, SSMS, SQL Developer, or any other SQL client.

Custom SQL Query Executor

The **Custom SQL Query** button opens a dedicated query window that lets you run any `SELECT` statement against the connected database and export the results.

Steps

1. Click **Custom SQL Query** in the main window (a connection must already be established).
2. Enter your SQL in the text area:

```
SELECT c.name, COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY total_orders DESC;
```

3. Select one or both export formats:
 - **Export as CSV** — comma-separated values
 - **Export as Excel** — `.xlsx` file (requires `pandas` and `openpyxl`)
4. Click **Browse** to choose the output file path.
5. Click **Execute & Export**.

A preview of the first 100 rows and a row count are shown before the file is written.

PDF Generation

PDF export uses one of two backends — DbDocGen detects which is available.

wkhtmltopdf (Recommended)

1. Install the Python wrapper: `pip install pdfkit`
2. Download and install the wkhtmltopdf binary from wkhtmltopdf.org
3. Ensure `wkhtmltopdf` is on your system `PATH`

WeasyPrint

1. Install: `pip install weasyprint`
2. On Windows, also install the [GTK3 Runtime](#)

If neither backend is installed, the **Export as PDF** checkbox is disabled. The HTML and Markdown output is always available regardless.

Command-Line / API Usage

You can drive DbDocGen from Python without the GUI:

```
from db_parser import DatabaseParser
from db_doc_generator import DatabaseDocGenerator
from db_html_generator import HTMLGenerator

# 1. Connect and parse
parser = DatabaseParser(
    db_type="postgresql", # postgresql | sqlserver | oracle | informix
    host="localhost",
    port=5432,
    database="mydb",
    username="myuser",
    password="mypassword",
    schema="public" # optional
)

schema_data = parser.parse_schema(
    include_tables=True,
    include_views=True,
    include_indexes=True,
    include_constraints=True,
    include_relationships=True
)

# 2. Generate Markdown
doc_gen = DatabaseDocGenerator(schema_data)
markdown = doc_gen.generate_documentation(include_er_diagram=True)

with open("mydb_documentation.md", "w", encoding="utf-8") as f:
    f.write(markdown)

# 3. Generate HTML and PDF
html_gen = HTMLGenerator("mydb_documentation.md")
html_file = html_gen.generate_html() # returns path to .html file
pdf_file = html_gen.generate_pdf() # returns path to .pdf file (if backend available)
```

SQL Server with Windows Authentication

```
parser = DatabaseParser(  
    db_type="sqlserver",  
    host="SQLSERVER01",  
    port=1433,  
    database="AdventureWorks",  
    use_windows_auth=True,  
    sqlserver_driver="17"  
)
```

Troubleshooting

“Test Connection” fails

- Confirm the host, port, and database name are correct.
- Check that the database service is running and reachable from your machine.
- Verify the credentials have at least `SELECT` privileges on system catalog tables.
- For SQL Server, ensure the correct ODBC Driver version is installed and matches the **ODBC Driver Version** field.

No tables appear in the output

- The connected user may lack permission to read system catalog / information schema views.
- For PostgreSQL, confirm the **Schema** field matches the schema that contains your tables (default: `public`).
- For SQL Server, confirm the schema matches (default: `dbo`).

PDF export is unavailable or fails

- Check that `wkhtmltopdf` is installed and on the system `PATH` (`wkhtmltopdf --version` in a terminal).
- Alternatively, install WeasyPrint (plus GTK3 on Windows).
- HTML and Markdown output is always generated regardless of PDF availability.

ER diagram is blank or missing

- The diagram requires at least two tables with a foreign key relationship between them.
- Make sure **Include ER Diagram** and **Include Relationships** are both checked.

Dataverse — “Authentication failed”

- Confirm the Client ID, Client Secret, and Tenant ID match the Azure AD app registration exactly.
- Ensure the app registration has the **Dynamics CRM user_impersonation** API permission granted.
- Confirm the application user in Dataverse has a security role assigned.
- See `DATAVERSE_SETUP.md` for full setup instructions.

Excel export fails

- Install the required packages: `pip install pandas openpyxl`

Module not found errors

- Run `pip install -r requirements.txt` from the project root to install all dependencies at once.